# SeisBench Model API

## Installing SeisBench

The recommended way is installation through pip. Simply run:
```
pip install seisbench
```

## Overview

SeisBench offers the abstract class `WaveformModel` that every SeisBench model should subclass. This class offers two core functions, `annotate` and `classify`. Both of the functions are automatically generated based on configurations and submethods implemented in the specific model.

The `SeisBenchModel` bridges the gap between the pytorch interface of the models and the obspy interface common in seismology. It automatically assembles obspy streams into pytorch tensors and reassembles the results into streams. It also takes care of batch processing. Computations can be run on GPU by simply moving the model to GPU.

The `annotate` function takes an obspy stream object as input and returns annotations as stream again. For example, for picking models the output would be the characteristic functions, i.e., the pick probabilities over time.

```python
stream = obspy.read("my_waveforms.mseed")
annotations = model.annotate(stream) # Returns obspy stream object with annotations
```

The `classify` function also takes an obspy stream as input, but in contrast to the `annotate` function returns discrete results. The structure of these results might be model dependent. For example, a pure picking model will return a list of picks, while a picking and detection model might return a list of picks and a list of detections.

```python
stream = obspy.read("my_waveforms.mseed")
outputs = model.classify(stream) # Returns a list of picks
print(outputs)
```

Both `annotate` and `classify` can be supplied with waveforms from multiple stations at once and will automatically handle the correct grouping of the traces. For details on how to build your own model with SeisBench, check the documentation of `WaveformModel`. For details on how to apply models, check out

the Examples.

## Loading Pretrained Models

For annotating waveforms in a meaningful way, trained model weights are required. SeisBench offers a range of pretrained model weights through a common interface. Model weights are downloaded on the first use and cached locally afterwards. For some model weights, multiple versions are available. For details on accessing these, check the documentation at `from_pretrained`.

```python
import seisbench.models as sbm

sbm.PhaseNet.list_pretrained() # Get available models
model = sbm.PhaseNet.from_pretrained("original") # Load the original model weights released by PhaseNet authors
```

Pretrained models can not only be used for annotating data, but also offer a great starting point for transfer learning.

## Speeding Up Model Application

When applying models to large datasets, run time is often a major concern. Here are a few tips to make your model run faster:

- **Run on GPU.** Execution on GPU is usually faster, even though exact speed-ups vary between models. However, we note that running on GPU is not necessarily the most economic option. For example, in cloud applications it might be cheaper (and equally fast) to pay for a handful of CPU machines to annotate a large dataset than for a GPU machine.

- **Use a large `batch_size`.** This parameter can be passed as an optional argument to all models. Especially on GPUs, larger batch sizes lead to faster annotations. As long as the batch fits into (GPU) memory, it might be worth increasing the batch size.

- **Compile your model (torch 2.0+).** If you are using torch in version 2.0 or newer, compile your model. It's as simple as running `model = torch.compile(model)`. The compilation will take some time but if you are annotating large amounts of waveforms, it should pay off quickly. Note that there are many options for compile that might influence the performance gains considerably.

- **Use asyncio interface.** Load data in parallel while executing the model using the asyncio interface, i.e., `annotate_asyncio` and `classify_asyncio`. This is usually substantially faster because data loading is IO-bound while the actual annotation is compute-bound.

- **Manual resampling.** While SeisBench can automatically resample the waveforms, it can be faster to do the resampling manually beforehand. SeisBench uses obspy routines for resampling, which (as of 2023) are not parallelised. Check the required sampling rate with `model.sampling_rate`. Alternative routines are available, e.g., in the Pyrocko library.

## Models Integrated into SeisBench

You don't have to build models from scratch if you don't want to. SeisBench integrates the following notable models from the literature for you to use. Again, as they inherit from the common SeisBench model interface, all these deep learning models are constructed through PyTorch. Where possible, the original trained weights are imported and made available. These can be accessed via the `from_pretrained` method.

| Integrated Model | Task |
|---|---|
| `BasicPhaseAE` | Phase Picking |
| `CRED` | Earthquake Detection |
| `DPP` | Phase Picking |
| `DepthPhaseNet` | Depth estimation from depth phases |
| `DepthPhaseTEAM` | Depth estimation from depth phases |
| `DeepDenoiser` | Denoising |
| `SeisDAE` | Denoising |
| `EQTransformer` | Earthquake Detection/Phase Picking |
| `GPD` | Phase Picking |
| `LFEDetect` | Phase Picking (Low-frequency earthquakes) |
| `OBSTransformer` | Earthquake Detection/Phase Picking |
| `PhaseNet` | Phase Picking |
| `PhaseNetLight` | Phase Picking |
| `PickBlue` | Earthquake Detection/Phase Picking |
| `Skynet` | Phase Picking |
| `VariableLengthPhaseNet` | Phase Picking |

Currently integrated models are capable of earthquake detection and phase picking, waveform denoising, depth estimation, and low-frequency earthquake phase picking. Furthermore, with SeisBench you can build ML models to perform general seismic tasks such as magnitude and source parameter estimation, hypocentre determination etc.

## Best Practices

- If the waveform data happen to be extremely small in scale (`<=1e-10`), there might be risk of numerical instability. It is acceptable to increase the value first (by multiplying a large number like `1e10`) before normalization or passing to the model.

- Although the seisbench model API will normalize the waveform for you, it is still highly suggested to apply normalization yourself. Since seisbench's normalization scheme uses an epsilon `(waveform - mean(waveform)) / (std(waveform) + epsilon)`, for extremely small values (such as `<=1e-10`), their normalization can destroy the signals in the waveform.

- The seisbench model API can process a stream of waveform data of arbitrary length. Hence, it is not necessary to segment the data yourself. In addition, you should not assume a stream of waveform can only contain one P-wave and one S-wave. It is the best to treat the stream like what it is: a stream of continuous data.